

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 3 – Mock Interview

Course Code:	CSA06	Course Title:	Design and Analysis of Algorithms
CO Assessed:	CO5 – Naturalization (BL5, BL6)	Assessment:	Assessment Tool 3 – Mock Interview
Weightage:	30% of CIA	Total Marks:	100
CO5 Statement:	<i>Analyze computational problems using backtracking techniques and complexity classes such as P, NP, NP-Hard, and NP-Complete for combinatorial problem solving.</i>		
Student Name:	J KARTHIK	Reg. No.:	192472136
Section / Batch:	CSE - AI / 2024	Date:	

Instructions to Students

- Attend the mock interview individually and answer all technical questions clearly.
- Explain the logic, algorithm design steps, and complexity analysis wherever required.
- Demonstrate understanding of backtracking, recursion, and combinatorial problem-solving approaches.
- Differentiate clearly between P, NP, NP-Hard, and NP-Complete problems with suitable examples.
- Maintain clear communication, confidence, and logical presentation throughout the interview.

Question Type	Focus Area	Questions
Mock Interview	Demonstrate problem-solving skills in backtracking techniques and complexity classes through technical discussion and algorithmic analysis.	Q1- Q10

SECTION A – Mock Interview

Code of Conduct

I (J KARTHIK / 192472136) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: J KARTHIK

RUBRICS FOR CALCULATION

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Technical Knowledge	25%	Demonstrates strong and accurate subject knowledge with in-depth understanding	Shows good knowledge with minor gaps	Basic knowledge with noticeable errors	Lacks fundamental technical knowledge
Problem Solving	25%	Solves problems logically and applies concepts effectively in real scenarios	Applies concepts with moderate accuracy	Limited problem-solving ability; requires guidance	Unable to solve or apply concepts
Analytical Thinking	15%	Analyzes questions critically and provides well structured, logical answers	Provides reasonable analysis with some structure	Superficial or partially structured responses	No analytical thinking demonstrated
Communication	15%	Communicates clearly, confidently, and professionally with proper terminology	Communicates well with minor hesitation	Communication lacks clarity or confidence	Unable to communicate effectively
Confidence	10%	Displays high confidence, positive attitude, and professional behavior throughout	Shows good confidence with minor issues	Low confidence; inconsistent behavior	Lacks confidence and professionalism
Response to Questions	10%	Responds accurately, promptly, and elaborates with relevant examples	Responds correctly but with limited depth	Partial responses; needs prompting	Unable to respond appropriately

Marks Evaluation:

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Technical Knowledge	25%				
Problem Solving	25%				
Analytical Thinking	15%				
Communication	15%				
Confidence	10%				
Response to Questions	10%				
Total Marks (100):					

Faculty In-charge	Course Coordinator	Head of Department
Name: _____	Name: _____	Name: _____
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

Backtracking & Complexity — Mixed Question Set

Q1. Define backtracking. How does it differ from brute-force search and dynamic programming?

Definition

Backtracking is a systematic, depth-first way of exploring a problem's solution space by building a candidate solution incrementally, one decision at a time. After each decision, the partial candidate is checked against the problem's constraints. As soon as it is clear that a partial candidate cannot be extended into a valid full solution, the algorithm abandons it — 'backtracks' — and undoes the last decision to try the next alternative. It can be viewed as a depth-first traversal of an implicit state-space tree, where entire subtrees are pruned the moment a constraint is violated.

How It Differs from Brute-Force and Dynamic Programming

Aspect	Backtracking	Brute-Force Search	Dynamic Programming
Core idea	Build a candidate step by step, check constraints as you go, abandon early on failure	Generate every possible candidate (often fully) and test each one afterwards	Break the problem into overlapping subproblems, solve each once, and reuse stored results
When constraints are checked	During construction, as early as possible	Only after a full candidate is generated	Not constraint-checking driven — driven by a recurrence relation
Wasted work	Pruned away as soon as a partial solution fails	None avoided — every full candidate is generated even if doomed early on	Avoided via memoisation / tabulation, not pruning
Typical use case	Constraint satisfaction: N-Queens, Sudoku, maze paths, subsets/permutations	Any exhaustive search where no early pruning is applied	Optimisation problems with optimal substructure, e.g., shortest path, knapsack, edit distance
Worst-case complexity	Same order as brute-force, but pruning usually reduces the practical search drastically	Exponential / factorial, no reduction	Often polynomial, because each distinct subproblem is solved only once

In short: backtracking is best understood as brute-force search made intelligent through early constraint checking and pruning — it still explores the same conceptual search space as brute force but skips large portions of it. Dynamic programming is a different strategy altogether: it applies only when subproblems overlap and recur, and instead of pruning a search tree it eliminates repeated computation by caching subproblem results, which is why DP can turn an exponential backtracking solution (like naive Fibonacci or naive knapsack) into a polynomial-time one when the overlapping-subproblem structure exists.

Q41. Design a backtracking algorithm for the N-Queens problem. Explain how conflicts are detected and avoided during placement.

Problem Definition

Place N queens on an N x N chessboard such that no two queens attack each other — i.e. no two queens share the same row, column, or diagonal.

State Representation

Since a valid solution must have exactly one queen per row, the board is represented not as an N x N grid but as an array `col[0..N-1]`, where `col[r]` is the column of the queen placed in row `r`. This built-in row constraint reduces the branching factor at each recursive level from N^2 (any cell) to N (any column in the current row).

Conflict Detection

Two queens at $(r1, c1)$ and $(r2, c2)$ conflict if any of the following hold:

- Same column: $c1 == c2$
- Same diagonal (top-left to bottom-right): $r1 - c1 == r2 - c2$
- Same anti-diagonal (top-right to bottom-left): $r1 + c1 == r2 + c2$

To make this an $O(1)$ check instead of scanning all previously placed queens, three boolean tracking arrays are maintained: `usedCol[N]` for columns, `usedDiag1[2N-1]` indexed by $(r - c + N - 1)$ for the ' $\backslash$ ' diagonals, and `usedDiag2[2N-1]` indexed by $(r + c)$ for the ' $/$ ' diagonals. A column or diagonal index becomes 'used' the moment a queen occupies it, and is freed again on backtrack.

Backtracking Algorithm

```
ALGORITHM SolveNQueens(N):
    col[N] = undefined for all rows
    usedCol[N] = FALSE, usedDiag1[2N-1] = FALSE, usedDiag2[2N-1] = FALSE
    PlaceQueen(0, N, col, usedCol, usedDiag1, usedDiag2)

FUNCTION PlaceQueen(row, N, col, usedCol, usedDiag1, usedDiag2):
    IF row == N:
        PRINT col                                // one full valid solution
        RETURN                                  // (or RETURN TRUE to stop at first solution)
    FOR c = 0 TO N-1:
        d1 = row - c + N - 1
        d2 = row + c
        IF NOT usedCol[c] AND NOT usedDiag1[d1] AND NOT usedDiag2[d2]:
            col[row] = c
            usedCol[c] = usedDiag1[d1] = usedDiag2[d2] = TRUE
            PlaceQueen(row + 1, N, col, usedCol, usedDiag1, usedDiag2)
            usedCol[c] = usedDiag1[d1] = usedDiag2[d2] = FALSE // backtrack
            col[row] = undefined
```

How Conflicts Are Avoided

Rather than placing all N queens and then checking the whole board for attacks (the brute-force approach), the algorithm checks safety before every placement using the three tracking arrays. This means an invalid branch is cut off at the exact row where the conflict would occur, so the algorithm never wastes time exploring the (large) subtree beneath a doomed partial placement. On backtrack, the same three arrays are reset for the undone queen, which is what allows the next candidate column to be tried cleanly in the next iteration of the loop.

Q81. Prove at a high level that the Hamiltonian Cycle problem is NP-Complete.

The Hamiltonian Cycle (HC) decision problem asks: given a graph $G = (V, E)$, does G contain a cycle that visits every vertex exactly once and returns to its starting vertex? To prove HC is NP-Complete, two things must be shown: (1) HC is in NP, and (2) HC is NP-hard.

Step 1 — HC is in NP

A problem is in NP if a proposed solution ('certificate') can be verified in polynomial time. For HC, the certificate is simply a proposed ordering of the vertices claimed to form a Hamiltonian cycle. Verification consists of two checks: (a) the sequence contains every vertex of G exactly once, and (b) every pair of consecutive vertices in the sequence (including the pair wrapping from the last vertex back to the first) is connected by an edge in G . Both checks take $O(V + E)$ time, which is polynomial in the size of the input graph, so HC is in NP.

Step 2 — HC is NP-hard

NP-hardness is shown by giving a polynomial-time reduction from a problem already known to be NP-complete into HC. The standard route (part of Karp's original 1972 chain of 21 NP-complete problems) reduces from Vertex Cover, which is itself already proven NP-complete via a reduction from 3-SAT:

- Start with a Vertex Cover instance: a graph G and integer k , asking whether a set of at most k vertices touches every edge.
- Construct a new graph G' by replacing each edge of G with a small gadget structure (a chain of 'crossover' vertices) and adding k additional 'selector' vertices representing the k cover slots.
- The gadgets are built so that any Hamiltonian cycle in G' is forced to pass through each edge-gadget in a way that corresponds exactly to that edge being 'covered' by one of its two endpoints being chosen.
- This construction can be built in time polynomial in the size of G and k (the gadgets are of fixed, small size per edge/vertex).
- The key correctness property of the reduction is: G' has a Hamiltonian cycle if and only if G has a vertex cover of size $\leq k$.

Because Vertex Cover is already known to be NP-complete, and this construction reduces it to Hamiltonian Cycle in polynomial time, HC inherits Vertex Cover's hardness — meaning every problem in NP can be reduced to HC (via Vertex Cover), so HC is NP-hard.

Conclusion

HC is in NP (Step 1) and HC is NP-hard (Step 2). By definition, a problem that is both in NP and NP-hard is NP-Complete. Therefore, the Hamiltonian Cycle problem is NP-Complete. (Note: this is a high-level sketch of the reduction's structure and correctness property — the full formal proof spells out the exact gadget graph and a detailed case-by-case correctness argument, typically found in Karp's original paper or standard algorithms textbooks such as Cormen et al.)

Q121. You are designing a scheduler for 100 university exams with multiple constraints. Which technique — backtracking, dynamic programming, or greedy — would you choose and why?

Nature of the Scenario

Exam scheduling is fundamentally a constraint satisfaction problem: each exam must be assigned a time slot (and typically a room/invigilator) such that hard constraints hold exactly — no student has two exams at the same time, no room exceeds its capacity, no faculty member is double-booked, and so on. Structurally this is very close to graph colouring (exams that share a student are 'edges' that cannot receive the same 'colour', i.e. time slot), which is a classic combinatorial constraint problem, not a simple numeric optimisation with a clean recursive formula.

Comparing the Three Techniques

Technique	Fit for this scenario
Dynamic Programming	Poor fit. DP needs a compact, low-dimensional state (like 'remaining capacity' in knapsack) with overlapping subproblems. The state here would need to track which of ~100 exams are already placed and every pairwise conflict among them — an exponential state space with no useful overlap to exploit, so DP does not scale down to something tractable.
Greedy	Fast and simple (e.g., a Welsh-Powell-style 'most-constrained exam first' heuristic), and scales easily to 100 exams. However, it cannot guarantee a fully constraint-satisfying schedule — a locally good choice early on can force a conflict later that greedy can never undo, since it never reconsiders past decisions.
Backtracking	Best fit. It explores exam-to-slot assignments incrementally and checks all hard constraints (student clashes, room capacity, faculty availability) at each assignment, backtracking the moment a choice creates a conflict. This guarantees a fully valid schedule will be found if one exists, unlike greedy.

Recommendation and Justification

Backtracking is the right choice, because in exam scheduling correctness of the hard constraints matters more than raw speed — an 'almost correct' timetable with one student double-booked is not a usable answer, whereas a slightly slower search that guarantees validity is exactly what a university needs. At a scale of 100 exams, plain backtracking would still be exponential in the worst case, so in practice it should be strengthened with constraint-propagation heuristics that are standard companions to backtracking:

- Minimum Remaining Values (MRV): schedule the most constrained exam (fewest legal remaining slots) first, so conflicts are discovered early rather than late.
- Forward checking: after each assignment, immediately eliminate now-illegal slot choices from the remaining unscheduled exams, shrinking future branching before it happens.
- A greedy ordering heuristic (e.g., schedule exams with the most student overlaps first) can be used just to choose the order in which backtracking tries exams — this hybrid gets greedy's speed benefit for guiding the search while keeping backtracking's correctness guarantee.

This combination — backtracking with propagation and a greedy variable-ordering heuristic — is, in fact, essentially what real university timetabling software (built on Constraint Programming engines) does under

the hood. Dynamic programming is set aside here because there is no compact recursive relation to exploit; a pure greedy approach is set aside because it cannot be trusted to produce a fully valid timetable without any fallback.

Q161. In your own words, explain the concept of NP-Completeness to a first-year student.

Think about a jigsaw puzzle. If I hand you a finished puzzle, you can glance at it and quickly tell whether the pieces actually fit together correctly — that check takes almost no time at all, even for a big puzzle. But if I instead hand you a box of scattered pieces and ask you to assemble it from scratch, that could take you a very long time, and the bigger the puzzle gets, the worse it gets — the number of ways to try fitting pieces together explodes.

That gap — 'quick to check, but seemingly slow to solve' — is exactly the idea behind a class of problems computer scientists call NP. A problem is in NP if, whenever someone hands you a proposed answer, you can verify whether it's correct quickly (technically, in time that grows only polynomially with the size of the problem).

Now, within NP there's an especially important group of problems called NP-Complete. These are the 'hardest' problems in NP, in a very specific sense: every other problem in NP can be quickly rephrased ('reduced') into any one of these NP-Complete problems. That means if anyone ever found a fast way to solve just one NP-Complete problem, that same trick — via the rephrasing step — would instantly give a fast solution to every problem in NP, including thousands of important real-world problems in scheduling, logistics, circuit design, and cryptography.

So calling a problem 'NP-Complete' is really making two claims at once: checking a proposed solution is easy, and solving it from scratch is (as far as anyone has ever proven) at least as hard as the hardest problems we know of in this class. Nobody has ever found a fast algorithm for any NP-Complete problem, but nobody has proven that a fast algorithm is impossible either — that open question is the famous 'P vs NP' problem, one of the biggest unsolved questions in computer science and mathematics. Most researchers believe P is not equal to NP (that these problems really are hard), but proving that rigorously is still an open challenge.

A concrete example a first-year student will recognise: Sudoku. Checking whether a completed 9x9 grid is a valid solution takes almost no time. But solving a hard Sudoku puzzle from a mostly-empty grid can take a lot of searching — and for a generalised $N \times N$ version of Sudoku, this solving problem is, in fact, known to be NP-Complete.